COMMUNITY CODE PACK • EPISODE 09

Real-Life JARVIS: Deep PC Diagnostics & Hardware Telemetry

Engineered by **Swayam Dhawale** | Published by **Pulse of AI**

STEP 1: REQUIRED PYTHON LIBRARIES

Run in your terminal:

```
pip install google-genai pyaudio pycaw screen-brightness-control psutil WMI rich python-dotenv websockets
```

STEP 2: API CREDENTIALS SETUP

Create a `.env` file in your project folder and add your key:

```
GOOGLE_API_KEY=your_actual_gemini_api_key_here
```

COMPLETE BUILD JOURNEY • EPISODES 1 TO 8

Part	Milestone & Topic	Direct YouTube Link
Part 1	Voice Architecture & Core Neural Handshake	Watch Episode 1
Part 2	Bilingual Activation & Low-Latency Stream	Watch Episode 2
Part 3	Custom Tone, Indian Accent & Persona Tuning	Watch Episode 3
Part 4	Real-Time Interruptions & Natural Turn-Taking	Watch Episode 4
Part 5	70+ Global Languages Unlocked	Watch Episode 5
Part 6	Autonomous Desktop Navigation & App Launcher	Watch Episode 6
Part 7	Multi-App Orchestration & Workflow Automation	Watch Episode 7
Part 8	Direct Hardware Access: Volume & Screen Brightness	Watch Episode 8

SOURCE FILE 01: HARDWARE_CONTROL.PY

📁 **hardware_control.py — Silicon Probing, Thermal Sensors & Rich Terminal HUD**

```
\"\"\"
hardware_control.py
Hardware Control, CPU/GPU Thermals & Sci-Fi HUD for JARVIS
Built by Swayam Dhawale, Founder of Pulse of AI.
\"\"\"

from pycaw.pycaw import AudioUtilities
import screen_brightness_control as sbc
import psutil
import subprocess
import wmi
from rich.console import Console
from rich.panel import Panel
from rich.table import Table

console = Console()

# --- Temperature & GPU Extraction Helpers ---

def get_cpu_temperature() -> float | None:
    \"\"\"Reads the current CPU temperature in Celsius via Windows WMI.\"\"\"
    try:
        w = wmi.WMI(namespace="root\\wmi")
        temperature_info = w.MSAcpi_ThermalZoneTemperature()
        if temperature_info:
```

```
            temp_kelvin_tenths = temperature_info[0].CurrentTemperature
            temp_celsius = round((temp_kelvin_tenths / 10.0) - 273.15, 1)
            if 0 < temp_celsius < 125:
                return temp_celsius
    except Exception:
        pass

    try:
        w_ohm = wmi.WMI(namespace="root\\OpenHardwareMonitor")
        sensors = w_ohm.Sensor()
        for sensor in sensors:
            if sensor.SensorType == "Temperature" and "CPU" in sensor.Name:
                return round(float(sensor.Value), 1)
    except Exception:
        pass

    return None

def get_nvidia_gpu_telemetry() -> tuple[str, str]:
    \"\"\"Reads temperature and utilization directly from nvidia-smi (Python 3.12 compatible).\"\"\"
    try:
        cmd = "nvidia-smi --query-gpu=temperature.gpu,utilization.gpu --format=csv,noheader,nounits"
        output = subprocess.check_output(cmd, shell=True, text=True).strip()
        if output:
            parts = [p.strip() for p in output.split(",")]
            temp = f\"{parts[0]}°C\"
            load = f\"{parts[1]}%\"
            return temp, load
    except Exception:
        pass
    return "Offline", "0%"

# --- Terminal Sci-Fi HUD ---

def render_hud_telemetry(cpu: float, cpu_temp: str, ram_pct: float, ram_used: float, ram_total: float,
                         battery_pct: int, charging: bool, disk_free: float, gpu_temp: str, gpu_load: str):
    \"\"\"Renders a sleek, futuristic terminal HUD table.\"\"\"
    table = Table(title="[bold cyan]⚡ JARVIS CORE TELEMETRY ⚡[/bold cyan]", border_style="cyan", show_lines=True)
    table.add_column("Subsystem", justify="left", style="white")
    table.add_column("Diagnostics / Metrics", justify="right", style="bold green")

    cpu_style = "bold red" if cpu > 80 else ("bold yellow" if cpu > 50 else "bold green")
    ram_style = "bold red" if ram_pct > 85 else "bold green"
    batt_style = "bold green" if (charging or battery_pct > 30) else "bold red"

    table.add_row("🧠 CPU Core Load & Temp", f"[{cpu_style}]{cpu}% | {cpu_temp}[/{cpu_style}]\")
    table.add_row("🎮 GPU Graphics Temp", f"[bold magenta]{gpu_temp} | Load: {gpu_load}[/bold magenta]\")
    table.add_row("💾 System Memory (RAM)", f"[{ram_style}]{ram_pct}% ({ram_used} GB / {ram_total} GB)[/{ram_style}]\")
    table.add_row("🔋 Arc Reactor (Battery)", f"[{batt_style}]{battery_pct}% ({'Charging ⚡' if charging else 'Discharging
▼ '})[/{batt_style}]\")
    table.add_row("💿 Primary Storage (Drive C)", f"[bold white]{disk_free} GB Free[/bold white]\")

    console.print(Panel(table, border_style="bold blue", expand=False))

# --- Hardware Audio Controls ---
def set_volume(level: int) -> str:
    \"\"\"Sets the Windows master audio volume level.\"\"\"
    try:
        target_level = max(0, min(100, int(level)))
        device = AudioUtilities.GetSpeakers()
        volume = device.EndpointVolume
        volume.SetMasterVolumeLevelScalar(target_level / 100.0, None)
        return f\"Volume set to {target_level}%\"
    except Exception as e:
        return f\"Failed to set volume: {str(e)}\"

def get_volume() -> str:
    \"\"\"Gets the current master audio volume percentage.\"\"\"
    try:
        device = AudioUtilities.GetSpeakers()
        volume = device.EndpointVolume
        current_volume = int(round(volume.GetMasterVolumeLevelScalar() * 100))
```

```
        return f\"The current audio volume is at {current_volume}%\"
    except Exception as e:
        return f\"Failed to read volume: {str(e)}\"

# --- Display Brightness Controls ---
def set_brightness(level: int) -> str:
    \"\"\"Sets the laptop display brightness.\"\"\"
    try:
        target_level = max(0, min(100, int(level)))
        sbc.set_brightness(target_level)
        return f\"Brightness set to {target_level}%\"
    except Exception as e:
        return f\"Failed to set brightness: {str(e)}\"

def get_brightness() -> str:
    \"\"\"Gets the current display brightness percentage.\"\"\"
    try:
        current_brightness = sbc.get_brightness()
        if isinstance(current_brightness, list):
            current_brightness = current_brightness[0]
        return f\"The current display brightness is at {current_brightness}%\"
    except Exception as e:
        return f\"Failed to read brightness: {str(e)}\"

# --- Battery Telemetry ---
def get_battery_status() -> str:
    \"\"\"Gets real-time battery percentage and charging state.\"\"\"
    try:
        battery = psutil.sensors_battery()
        if battery is None:
            return \"No battery detected on this system, sir. Operating on continuous AC power.\"

        percent = int(battery.percent)
        is_charging = battery.power_plugged
        charging_text = \"plugged in and charging\" if is_charging else \"discharging on battery power\"
        return f\"The battery is currently at {percent}%, {charging_text}.\"
    except Exception as e:
        return f\"Failed to retrieve battery telemetry: {str(e)}\"

# --- Temperature Tool ---
def get_system_temperatures() -> str:
    \"\"\"Returns real-time thermal readings for both CPU and GPU.\"\"\"
    cpu_t = get_cpu_temperature()
    gpu_temp, _ = get_nvidia_gpu_telemetry()
    cpu_report = f\"{cpu_t}°C\" if cpu_t is not None else \"Nominal (Sensor standby)\"
    return f\"CPU temperature is at {cpu_report}, and the GPU is running at {gpu_temp}, sir.\"

# --- Full Telemetry & Diagnostics with Thermals ---
def get_system_vitals() -> str:
    \"\"\"Collects CPU, RAM, GPU, Disk, Battery, and Thermals while rendering a sci-fi HUD.\"\"\"
    try:
        cpu_usage = psutil.cpu_percent(interval=0.1)
        mem = psutil.virtual_memory()
        mem_used_gb = round(mem.used / (1024 ** 3), 1)
        mem_total_gb = round(mem.total / (1024 ** 3), 1)
        mem_percent = mem.percent

        disk = psutil.disk_usage(\"C:\\\\\")
        disk_free_gb = round(disk.free / (1024 ** 3), 1)

        battery = psutil.sensors_battery()
        battery_pct = int(battery.percent) if battery else 100
        is_charging = battery.power_plugged if battery else True

        cpu_temp_val = get_cpu_temperature()
        cpu_temp_str = f\"{cpu_temp_val}°C\" if cpu_temp_val is not None else \"Normal\"
        gpu_temp_str, gpu_load_str = get_nvidia_gpu_telemetry()

        render_hud_telemetry(
            cpu=cpu_usage,
            cpu_temp=cpu_temp_str,
            ram_pct=mem_percent,
```

```
            ram_used=mem_used_gb,
            ram_total=mem_total_gb,
            battery_pct=battery_pct,
            charging=is_charging,
            disk_free=disk_free_gb,
            gpu_temp=gpu_temp_str,
            gpu_load=gpu_load_str
        )

        response_summary = (
            f\"CPU is running at {cpu_usage}% load and {cpu_temp_str}. \"
            f\"GPU temperature is at {gpu_temp_str} with {gpu_load_str} load. \"
            f\"RAM is at {mem_percent}% usage. \"
            f\"Battery is at {battery_pct}%, {'charging' if is_charging else 'on battery'}. \"
            f\"Thermals and performance remain optimal, sir.\"
        )
        return response_summary
    except Exception as e:
        return f\"Failed to retrieve system vitals: {str(e)}\"
```

SOURCE FILE 02: AGENT.PY

📁 **agent.py — Gemini Live Multimodal WebSocket & Subsystem Tool Execution**

```
import asyncio
import datetime
import os
import subprocess
import sys
import webbrowser
import pyaudio
import winsound
from dotenv import load_dotenv
from google import genai
from google.genai import errors, types
import websockets
from rich.console import Console

from hardware_control import (
    get_brightness,
    get_volume,
    set_brightness,
    set_volume,
    get_battery_status,
    get_system_vitals,
    get_system_temperatures,
)

load_dotenv()

console = Console()
client = genai.Client(api_key=os.getenv("GOOGLE_API_KEY"))

FORMAT = pyaudio.paInt16
CHANNELS = 1
RATE_IN = 16000
RATE_OUT = 24000
CHUNK = 512

p = pyaudio.PyAudio()

model_is_speaking = False

def play_sfx_chime():
    try:
        winsound.Beep(1200, 60)
        winsound.Beep(1800, 80)
    except Exception:
        pass
```

```
def play_startup_chime():
    try:
        winsound.Beep(600, 70)
        winsound.Beep(900, 70)
        winsound.Beep(1400, 120)
    except Exception:
        pass

def open_website(url: str) -> str:
    clean_url = url.strip()
    if not clean_url.startswith(("http://", "https://")):
        clean_url = "https://" + clean_url
    webbrowser.open(clean_url)
    return f\"Opened {clean_url}\"

def open_application(app_name: str) -> str:
    query = app_name.lower().strip()
    uwp_protocols = {
        "camera": "microsoft.windows.camera:",
        "calculator": "calculator:",
        "settings": "ms-settings:",
        "spotify": "spotify:",
        "paint": "mspaint",
        "notepad": "notepad",
        "cmd": "cmd",
        "terminal": "wt",
        "task manager": "taskmgr",
    }

    if query in uwp_protocols:
        try:
            target = uwp_protocols[query]
            if target.endswith(":"):
                os.system(f\"start {target}\")
            else:
                subprocess.Popen(target, shell=True)
            return f\"Launched {app_name}\"
        except Exception:
            pass

    if sys.platform == "win32":
        search_dirs = [
            r"C:\ProgramData\Microsoft\Windows\Start Menu\Programs",
            os.path.expandvars(r"%APPDATA%\Microsoft\Windows\Start Menu\Programs"),
            os.path.expandvars(r"%LOCALAPPDATA%\Programs"),
        ]
        for directory in search_dirs:
            if not os.path.exists(directory):
                continue
            for root, _, files in os.walk(directory):
                for file in files:
                    if file.lower().endswith((".lnk", ".exe")):
                        file_name = file.lower().rsplit(".", 1)[0]
                        if query == file_name:
                            os.startfile(os.path.join(root, file))
                            return f\"Launched {app_name}\"
    return f\"Executed {app_name}\"

tools = [
    open_website,
    open_application,
    set_volume,
    get_volume,
    set_brightness,
    get_brightness,
    get_battery_status,
    get_system_vitals,
    get_system_temperatures,
]

def get_time_greeting() -> str:
    current_hour = datetime.datetime.now().hour
```

```
    if 5 <= current_hour < 12:
        return "Good morning, sir."
    elif 12 <= current_hour < 17:
        return "Good afternoon, sir."
    else:
        return "Good evening, sir."

async def send_audio(session):
    global model_is_speaking
    mic_stream = p.open(format=FORMAT, channels=CHANNELS, rate=RATE_IN, input=True, frames_per_buffer=CHUNK)
    console.print("[cyan]🎤 [LISTENING] Multilingual Voice Pipeline Ready...[/cyan]")
    try:
        while True:
            data = await asyncio.to_thread(mic_stream.read, CHUNK, exception_on_overflow=False)
            if model_is_speaking:
                continue
            await session.send_realtime_input(audio={"data": data, "mime_type": "audio/pcm;rate=16000"})
    except asyncio.CancelledError:
        raise
    finally:
        mic_stream.stop_stream()
        mic_stream.close()

async def receive_audio(session, audio_queue):
    global model_is_speaking
    try:
        while True:
            async for response in session.receive():
                if response.tool_call is not None:
                    function_responses = []
                    for call in response.tool_call.function_calls:
                        await asyncio.to_thread(play_sfx_chime)
                        result = ""
                        if call.name == "open_website":
                            result = open_website(call.args.get("url", "https://google.com"))
                        elif call.name == "open_application":
                            result = open_application(call.args.get("app_name", ""))
                        elif call.name == "set_volume":
                            result = set_volume(call.args.get("level", 50))
                        elif call.name == "get_volume":
                            result = get_volume()
                        elif call.name == "set_brightness":
                            result = set_brightness(call.args.get("level", 50))
                        elif call.name == "get_brightness":
                            result = get_brightness()
                        elif call.name == "get_battery_status":
                            result = get_battery_status()
                        elif call.name == "get_system_vitals":
                            result = get_system_vitals()
                        elif call.name == "get_system_temperatures":
                            result = get_system_temperatures()

                        console.print(f\"\\n[bold yellow]⚙️ [TOOL EXECUTED: {call.name}][/bold yellow] [green]{result}[/gr
een]\")
                        function_responses.append(types.FunctionResponse(name=call.name, id=call.id, response={"result": re
sult}))
                    if function_responses:
                        await session.send_tool_response(function_responses=function_responses)

                server_content = response.server_content
                if server_content is not None:
                    if server_content.interrupted:
                        while not audio_queue.empty():
                            audio_queue.get_nowait()
                        model_is_speaking = False
                    if server_content.input_transcription:
                        console.print(f\"\\r[bold white][USER][/bold white] {server_content.input_transcription.text}\")
                    model_turn = server_content.model_turn
                    if model_turn is not None:
                        if not model_is_speaking:
                            model_is_speaking = True
                            console.print("[bold cyan]🔊 [JARVIS ANSWERING...][/bold cyan]")
```

```
                        for part in model_turn.parts:
                            if part.inline_data and part.inline_data.data:
                                await audio_queue.put(part.inline_data.data)
                    if server_content.turn_complete:
                        model_is_speaking = False
                        console.print("[cyan]🎤 [LISTENING] Awaiting instructions...[/cyan]")
    except asyncio.CancelledError:
        raise
    finally:
        await audio_queue.put(None)

async def play_audio(audio_queue):
    speaker_stream = p.open(format=FORMAT, channels=CHANNELS, rate=RATE_OUT, output=True, frames_per_buffer=CHUNK)
    try:
        while True:
            audio_data = await audio_queue.get()
            if audio_data is None:
                break
            await asyncio.to_thread(speaker_stream.write, audio_data)
    except asyncio.CancelledError:
        raise
    finally:
        speaker_stream.stop_stream()
        speaker_stream.close()

async def run_session(config, model_id, greeting):
    async with client.aio.live.connect(model=model_id, config=config) as session:
        console.print("[bold green]✓ Secure Neural Bridge Established with Gemini Live.[/bold green]")
        await asyncio.to_thread(play_startup_chime)
        audio_queue = asyncio.Queue()
        send_task = asyncio.create_task(send_audio(session))
        receive_task = asyncio.create_task(receive_audio(session, audio_queue))
        playback_task = asyncio.create_task(play_audio(audio_queue))
        await session.send_realtime_input(text=f\"Say aloud: 'All protocols initialized. {greeting}'\")
        try:
            await asyncio.gather(send_task, receive_task, playback_task)
        finally:
            send_task.cancel()
            receive_task.cancel()
            playback_task.cancel()
            await asyncio.gather(send_task, receive_task, playback_task, return_exceptions=True)

async def main():
    model_id = "gemini-3.1-flash-live-preview"
    greeting = get_time_greeting()
    config = types.LiveConnectConfig(
        response_modalities=["AUDIO"],
        tools=tools,
        speech_config=types.SpeechConfig(voice_config=types.VoiceConfig(prebuilt_voice_config=types.PrebuiltVoiceConfig(voi
ce_name="Puck"))),
        system_instruction=types.Content(parts=[types.Part.from_text(text=(
            "You are JARVIS created by Swayam Dhawale, Founder of Pulse of AI. "
            "Speak in an articulate, polite Indian accent. Always address user as 'sir'. "
            "Call tools when asked about apps, volume, brightness, battery, and diagnostics vitals."
        ))])
    )
    while True:
        try:
            await run_session(config, model_id, greeting)
        except Exception as e:
            console.print(f\"\\n[bold red]🔄 Connection dropped ({e}). Reconnecting in 1 second...[/bold red]\")
            await asyncio.sleep(1)

if __name__ == "__main__":
    if sys.platform == "win32":
        asyncio.set_event_loop_policy(asyncio.WindowsProactorEventLoopPolicy())
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        console.print(\"\\n[bold red]🔴 JARVIS session terminated by user.[/bold red]\")
```

```
    finally:
        p.terminate()
```